

ATmega2560: AVRDUDE + Atmel-ICE

Linux setup, ISP wiring, timestamped memory backups, firmware flashing, and verification. Commands below target the ATmega2560 using the Atmel-ICE AVR ISP interface.

1. Install and identify the programmer

```
sudo apt update
sudo apt install avrdude
avrdude -?
avrdude -c ?
lsusb
```

Confirm that **atmelice_isp** appears in the supported programmer list and that USB detects the Atmel-ICE. If the distro package lacks support, install a newer AVRDUDE release. For USB access errors, test once with `sudo`, then configure a udev rule using the actual USB vendor/product IDs shown by `lsusb`.

2. Connect Atmel-ICE in ISP mode

Atmel-ICE ISP signal	ATmega2560 signal
MISO	PB3
MOSI	PB2
SCK	PB1
RESET	RESET
VTG / target voltage sense	Target VCC
GND	Ground

The Atmel-ICE normally does not power the target. Power the ATmega2560 separately and share ground. On an Arduino Mega 2560, use the 6-pin ICSP header. Verify connector orientation and target voltage before connecting.

3. Test target communication

```
avrdude -c atmelice_isp -p m2560
# If necessary, use a slower ISP clock:
avrdude -c atmelice_isp -p m2560 -B 10
```

Expected ATmega2560 signature: **0x1E9801**. ISP needs a working target clock. If the signature is wrong, check target power, RESET, wiring, clock settings, and ISP speed.

4. Read Flash, EEPROM, fuses, lock bits and signature

Run **backup_atmega2560.sh**. It creates a timestamped subfolder containing seven timestamped files and a SHA-256 checksum manifest. It uses only read operations.

```
./backup_atmega2560.sh
# Optional output parent directory:
./backup_atmega2560.sh /path/to/backups
```

Individual read operations use **-U memory:r:filename:format**. For example:

```
avrdude -c atmelice_isp -p m2560 \
-U flash:r:flash.hex:i \
-U eeprom:r:eeprom.hex:i \
-U lfuse:r:lfuse.hex:h \
-U hfuse:r:hfuse.hex:h \
```

```
-U efuse:r:efuse.hex:h \  
-U lock:r:lock.hex:h \  
-U signature:r:signature.hex:h
```

Formats: **i** = Intel HEX, **h** = hex text, **r** = raw binary. Flash capacity: 256 KiB; EEPROM capacity: 4 KiB. Lock bits may prevent reading protected contents. Do not erase a locked chip when preservation is the goal.

5. Flash and verify a firmware HEX file

First make a backup. The flash script requires typing FLASH before programming and performs an explicit verify operation after writing.

```
./flash_verify_atmega2560.sh firmware.hex  
# Equivalent AVRDUDE operations:  
avrdude -c atmelice_isp -p m2560 \  
-U flash:w:firmware.hex:i \  
-U flash:v:firmware.hex:i
```

CAUTION: Flash programming can erase existing Flash, including a bootloader. A chip erase may erase EEPROM unless the EESAVE fuse is programmed. Do not assume fuses or EEPROM are restored by writing a firmware HEX file.

6. Verify without writing

```
./verify_atmega2560.sh firmware.hex  
# Equivalent:  
avrdude -c atmelice_isp -p m2560 -U flash:v:firmware.hex:i
```

7. Script setup and troubleshooting

```
chmod +x backup_atmega2560.sh flash_verify_atmega2560.sh verify_atmega2560.sh  
# Inspect an EEPROM binary backup (if read in raw format):  
xxd eeprom.bin  
# Validate saved checksum manifest:  
sha256sum -c checksums.sha256
```

Common issues: USB access denied (udev permissions); programmer not found (USB cable, AVRDUDE version); initialization/signature failure (target supply, common ground, ISP pin mapping, reset, clock, or ISP clock speed); all-FF Flash (erased device or protected read).

Quick reference

Read: -U flash:r:flash.hex:i **Write:** -U flash:w:firmware.hex:i **Verify:** -U flash:v:firmware.hex:i

AVRDUDE documentation: <https://avrdudes.github.io/avrdude/> | ATmega2560 device datasheet: Microchip.